МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ 3

Работа со строками в Python

Вариант 13

по дисциплине «Программирование на языке Python»

Выполнил
Тебаев Мансур Дахирович
Группа ЭНЭ-б-о-25-2
Проверил _____

Ставрополь 2026

1 Тема и цель работы

Тема — Работа со строками в Python. Освоить индексацию, срезы и методы строк, поиск символов и преобразование текста с сохранением заданных свойств результата.

Во всех обязательных группах индивидуальных заданий выбран вариант 13. Количество групп: 3.

Локальный проект: python-lab-03. Подготовлен локальный Git репозиторий; удалённый адрес будет добавлен после публикации владельцем. Ссылка на удалённый репозиторий: не опубликован.

2 Среда и порядок запуска

Проверка выполнена в Windows 10, Python 3.14.6, с использованием виртуального окружения. Сторонние библиотеки для учебных программ не требуются. Числа вводятся в десятичной записи с точкой; если иное не указано, поля вводятся отдельно, каждое с новой строки. Для списков вводится одна строка чисел через пробел.

Запуск и пошаговая отладка выполнены в VS Code 1.144.0 с Python Debugger 2026.6.0 через Extension Host API. Контрольные входы поданы из подготовленных данных. Отладчик подтвердил значения переменных и следующий шаг; журнал сохранён в results/vscode_debug.json.

Из папки проекта для создания собственного окружения и запуска используются команды:

```
python -m venv .venv
.\.venv\Scripts\python.exe --version
.\.venv\Scripts\python.exe replace_spaces.py
.\.venv\Scripts\python.exe verify.py
```

Файл verify.py использует тот интерпретатор, которым он запущен. Эталонные данные лежат в fixtures.json, полные результаты — в results/checks.json и results/checks.txt. Код 0 ожидается для корректного ввода, код 1 — для проверяемых ошибок диапазона или формата. Импорт учебных модулей не запускает диалог ввода благодаря защите __name__.

3 Разобранные примеры

Замена пробелов

Ввести строку и заменить каждый обычный пробел подчёркиванием. Пустая строка допустима.

replace изменяет все вхождения U+0020 и возвращает новую строку. Табуляции сохраняются. repr делает видимыми крайние пробелы, пустую строку и управляющие символы.

Файл полного решения: python-lab-03/replace_spaces.py.

```
def main():
    text = input()
    print(repr(text.replace(' ', '_')))


if __name__ == '__main__':
    main()
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
'мама мыла раму'	'мама_мыла_раму'	'мама_мыла_раму'
' а б '	'_а__б_'	'_а__б_'
''	''	''
'а\tб'	'а\tб'	'а\tб'

Удаление середины слова

Из непустого слова удалить средний символ при нечётной длине либо два средних символа при чётной. Для пустого ввода вывести ошибку и завершиться с кодом 1.

Правая сохраняемая часть начинается с middle + 1. Левая заканчивается перед middle для нечётной длины и перед middle − 1 для чётной. Из слов длины 1 и 2 получается пустая строка.

Файл полного решения: python-lab-03/remove_middle.py.

```
import sys


def main():
    word = input()
    if not word:
        print('Ошибка: пустое слово', file=sys.stderr)
        return 1
    middle = len(word) // 2
    left = middle if len(word) % 2 else middle - 1
    result = word[:left] + word[middle + 1:]
    print(repr(result))
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
кот	'кт'	'кт'
мама	'ма'	'ма'
я	''	''
мы	''	''
Пустая строка	Ошибка: пустое слово Код 1	Ошибка: пустое слово Код 1

Равномерное распределение пробелов

Ввести строку из двух или более слов и целую требуемую длину L. Слова разделены обычными пробелами, пробелов по краям и других пробельных символов нет. Исходная длина ≤ L ≤ 10000. Сохранить слова и

порядок; распределить пробелы так, чтобы ширины промежутков отличались не более чем на 1, более длинные промежутки находились левее. При равной длине допускается перераспределение существующих пробелов.

После split подсчитывается длина слов letters. Всего нужно L − letters пробелов и len(words) − 1 промежутков. divmod даёт базовую ширину и число добавочных пробелов. Первые extra промежутков получают base + 1 пробелов, остальные — base. join соединяет подготовленные части без дополнительного разделителя.

Файл полного решения: python-lab-03/justify_text.py.

```
import sys


def main():
    text = input()
    length = int(input())
    if (not text or text.startswith(' ') or text.endswith(' ')
            or any(c.isspace() and c != ' ' for c in text)):
        print('Ошибка: формат строки', file=sys.stderr)
        return 1
    words = text.split()
    if len(words) < 2 or not len(text) <= length <= 10000:
        print('Ошибка: слова или длина', file=sys.stderr)
        return 1
    letters = 0
    for word in words:
        letters += len(word)
    gap_count = len(words) - 1
    base, extra = divmod(length - letters, gap_count)
    parts = []
    for i, word in enumerate(words):
        parts.append(word)
        if i < gap_count:
            parts.append(' ' * (base + (i < extra)))
    result = ''.join(parts)
```

```
    print(repr(result))
    print(f'Длина: {len(result)}')
    return 0


if __name__ == '__main__':
    sys.exit(main())
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
'мама мыла раму'; L=17	'мама мыла раму' Длина: 17	'мама мыла раму' Длина: 17
'а б в'; L=7	'а б в' Длина: 7	'а б в' Длина: 7
'два слова'; L=9	'два слова' Длина: 9	'два слова' Длина: 9
'два слова'; L=6	Ошибка: слова или длина Код 1	Ошибка: слова или длина Код 1
'слово'; L=8	Ошибка: слова или длина Код 1	Ошибка: слова или длина Код 1
' два слова'; L=10	Ошибка: формат строки Код 1	Ошибка: формат строки Код 1
' '; L=12	Ошибка: формат строки Код 1	Ошибка: формат строки Код 1
'а\tб'; L=10	Ошибка: формат строки Код 1	Ошибка: формат строки Код 1
а б в; 6	'а б в' Длина: 6	'а б в' Длина: 6

4 Индивидуальные задания

Индивидуальное задание 1 вариант 13

Дано предложение. Вывести порядковые номера всех букв «и», находящихся на чётных местах. Каждый номер вывести с новой строки.

Сравнивается строчная кириллическая «и», регистр учитывается. enumerate(text, start=1) сразу даёт порядковые номера, поэтому условие position % 2 == 0 относится к местам 2, 4, 6 и далее. Индекс Python для такого символа на единицу меньше. Если результата нет, выводится «Не найдено». Пустое предложение допустимо; длина входной строки не больше 1000.

Файл полного решения: python-lab-03/individual_01.py.

```
def main():
    text = input()
    positions = []
    for position, character in enumerate(text, start=1):
        if position % 2 == 0 and character == 'и':
            positions.append(position)
    if positions:
        for position in positions:
            print(position)
    else:
        print('Не найдено')


if __name__ == '__main__':
    main()
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
''	Не найдено	Не найдено
'мир'	2	2

Ввод или сценарий	Ожидается	Получено
'иии'	2 4	2 4
'Ииi'	2 4	2 4
'иabc'	Не найдено	Не найдено
'аии'	2 4	2 4

Индивидуальное задание 2 вариант 13

Дано предложение, в котором имеются буквы «н» и «к». Определить, какая из этих букв встречается раньше при просмотре слева направо.

Предусловие: присутствуют обе строчные кириллические буквы, длина не больше 1000. find возвращает индекс первого вхождения. Меньший индекс соответствует более ранней букве. Проверка отсутствия не нужна при принятом предусловии; без него значение −1 потребовало бы отдельной обработки.

Файл полного решения: python-lab-03/individual_02.py.

```
def main():
    text = input()
    n_position = text.find('н')
    k_position = text.find('к')
    # По условию обе строчные кириллические буквы присутствуют.
    if n_position < k_position:
        print('Раньше н')
    else:
        print('Раньше к')


if __name__ == '__main__':
    main()
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
нк	Раньше н	Раньше н
кн	Раньше к	Раньше к
аник	Раньше н	Раньше н
ккнн	Раньше к	Раньше к
НКнк	Раньше н	Раньше н

Индивидуальное задание 3 вариант 13

Дано слово. Перенести его первую букву в конец слова.

Предусловие: непустое слово из русских или латинских букв, длина не больше 1000. Срез word[1:] содержит все символы после первого; к нему приписывается word[0]. Регистр и порядок остальных букв сохраняются. Слово из одной буквы не меняется.

Файл полного решения: python-lab-03/individual_03.py.

```
def main():
    word = input()
    # По условию слово непустое.
    result = word[1:] + word[0]
    print(repr(result))


if __name__ == '__main__':
    main()
```

Проверки программы. Полные входные данные, stdout, stderr и коды завершения сохранены в results/checks.txt.

Ввод или сценарий	Ожидается	Получено
я	'я'	'я'

Ввод или сценарий	Ожидается	Получено
мы	'ым'	'ым'
питон	'итонп'	'итонп'
Python	'ythonP'	'ythonP'
aaa	'aaa'	'aaa'

5 Трассировка и воспроизведение в редакторе

Для строки «мама мыла раму» и L = 17 длина слов 12, промежутков 2, пробелов 5. base = 2, extra = 1. Состояния после добавления каждого слова и его промежутка:

i	Слово	Ширина промежутка	parts после шага
0	мама	3	['мама', ' ']
1	мыла	2	['мама', ' ', 'мыла', ' ']
2	раму	Нет	['мама', ' ', 'мыла', ' ', 'раму']

Итоговая длина 4 + 4 + 4 + 3 + 2 = 17. Разность ширин 3 − 2 = 1, более длинный промежуток расположен слева. Перед parts.append с пробелами при i = 0 уже добавлено слово «мама», base = 2 и extra = 1. Следующий шаг добавляет три пробела. Эти значения подтверждены трассировкой Python.

В VS Code остановка на строке 23 justify_text.py при i = 0 показала base = 2, extra = 1, parts = [«мама»]. После одного шага к parts добавлены три пробела. Контрольный запуск выдал строку длиной 17. Значения совпали с трассировочной таблицей. Все проверки verify.py также успешно выполнены запуском из VS Code. Журнал остановок и вывода сохранён в results/vscode_debug.json.

6 Ответы на вопросы для защиты

1. Строка и байты. str — неизменяемая последовательность кодовых точек Unicode. bytes хранит байты; кодирование encode и декодирование decode связывают эти представления.

2. Литералы. Обычный литерал обрабатывает escape последовательности, сырой r сохраняет обратные косые черты с синтаксическими ограничениями, тройные кавычки допускают несколько строк.

3. Операции над строками. + соединяет, * повторяет целое число раз, in и not in проверяют наличие подстроки.

4. Индексы. Положительные индексы начинаются с 0, отрицательные отсчитываются с конца: −1 — последний. Недопустимый одиночный индекс вызывает IndexError.

5. Срез. s[start:stop:step] выбирает символы до исключённой границы stop. Границы срезов могут выходить за длину и корректируются; шаг не равен нулю.

6. Неизменяемость. Символы существующей строки изменить нельзя. s = s.replace(...) допустимо: имя связывается с новым результатом.

7. capitalize и title. capitalize делает первый символ заглавным и остальные строчными, title обрабатывает начала слов по правилам метода. Эти правила не учитывают все языковые нормы и собственные имена.

8. in и find. in нужен для ответа да или нет. find возвращает позицию первого вхождения либо −1, когда требуется место в строке.

9. Четыре метода поиска. find и rfind ищут первое и последнее вхождения и дают −1 при отсутствии; index и rindex вместо −1 вызывают ValueError.

10. Длина. len считает кодовые точки, а не байты и не всегда видимые знаки: буква с комбинируемым знаком может занимать несколько кодовых точек.

11. count. count считает неперекрывающиеся вхождения. Для пустой подстроки без ограничений результат равен len(s) + 1.

12. Форматированная строка. f-строка подставляет выражения в фигурных скобках. Буквальные скобки записывают как {{ и }}.

13. Ограничение поиска. Параметры start и end ограничивают рассматриваемый участок аналогично срезу. Возвращённый индекс относится к исходной строке.

14. format и спецификации. format и f-строки задают представление значения: например, .2f — два знака после точки, >10 — поле шириной не менее десяти с выравниванием вправо.

15. Проверки числовых символов. isdecimal проверяет десятичные цифры; isdigit дополнительно допускает некоторые цифры вроде надстрочных; isnumeric включает и другие числовые символы. Ни один метод не распознаёт произвольное число со знаком и точкой.

16. split. split() объединяет серии пробельных символов и убирает крайние пустые части; split(" ") разделяет только по обычному пробелу, сохраняя пустые части. Для пустой строки результаты [] и [""].

17. islower. Метод истинен, когда есть хотя бы один символ с регистром и все такие символы строчные. Для одних цифр результат False.

18. Первый символ. Безопасная проверка: bool(s) and s[0].isupper(). Сокращённое вычисление предотвращает индексирование пустой строки.

19. Строка и число. Сложение str и int не определено. Для текста используют str(number), f-строку или format; для арифметики преобразуют текст в число.

20. Разворот. s[::-1] выбирает символы с конца с шагом −1. Нулевой шаг не определяет продвижение и вызывает ValueError.

21. join. Разделитель.join(iterable) вставляет один разделитель между строковыми элементами. Все элементы должны быть строками; числа нужно предварительно преобразовать.

22. casefold. casefold предназначен для сравнения без учёта регистра Unicode и может преобразовывать сильнее, чем lower; например, ß становится ss.

23. Первый и последний символы. Сначала проверяют пустую строку, затем отдельно длину 1. Иначе соединяют преобразованный первый символ, s[1:-1] и преобразованный последний.

24. isupper. Нужен хотя бы один символ с регистром; все такие символы должны быть заглавными. Цифры и пунктуация не нарушают условие, но сами его не обеспечивают.

25. splitlines. Метод делит по различным границам строк. keepends=True сохраняет завершающие разделители в элементах результата.

26. replace с ограничением. s.replace(old, new, count) заменяет не более count вхождений слева направо. Без count заменяются все.

27. Начала и окончания. startswith и endswith проверяют начало и конец. removeprefix и removesuffix удаляют одну заданную подстроку на соответствующем краю, если она есть.

28. Пробельные символы. Обычный пробел — U+0020. isspace распознаёт также табуляции, переводы строк и другие пробельные кодовые точки; для пустой строки он ложен.

29. strip. strip(chars) удаляет с обоих краёв любые символы из заданного набора, а не точный префикс или суффикс.

30. expandtabs. Метод заменяет табуляцию пробелами до следующей позиции табуляции, зависящей от текущего столбца и tabsize. Перевод строки сбрасывает столбец.

31. partition. При найденном разделителе возвращается кортеж (до, разделитель, после), при отсутствии — (вся строка, "", ""). Пустой разделитель недопустим.

32. Тесты строк. Нужны пустая и минимальная допустимая строки, чётная и нечётная длины, отсутствие и повторы искомого, совпадения на

первом и последнем месте, различие регистра и похожие символы разных алфавитов.

7 Вывод

Разработаны и проверены 6 учебных программ. Успешно пройдены все 34 проверок, включая предусмотренные ошибочные входы. Подтверждены граничные условия, порядок обработки данных и соответствие фактических результатов ожидаемым. Закреплены срезы, строковые методы, различие индекса и порядкового номера, а также проверка длины и распределения пробелов.

8 Источники

Методические материалы курса. Лабораторная работа № 3. Работа со строками в Python. Общая папка: https://disk.yandex.ru/d/yT9RQ8LG9aQ_UA

Python 3.14. Стандартные типы и управление потоком выполнения: https://docs.python.org/3.14/library/stdtypes.html и https://docs.python.org/3.14/tutorial/controlflow.html